Module 2

Sometimes I get the urge to design a programming language. Then I just lay down until it goes away. - L. Peter Deutsch

Definitions

Alphabet: finite set of symbols

String (AKA lexeme): a sequence of symbols from the alphabet

Language: a set of strings

Syntax: The set of rules that specify the set of strings in the language. In our context, these strings are the set of legal looking programs.

Semantics: the set of rules that specify the meaning of each string in a language. In our context, these rules specify what programs are legal, and what a program does when it executes.

Two formal definitions of language theory appear here

Regular Expressions (REs): Rules that recognize tokens. Cannot specify which tokens are legal next to each other

The Scanner or Lexer is the part of the compiler which recognizes tokens according to RE's.

It is a **Deterministic Finite Automaton** (DFA).

Context Free Grammar (CFG): Rules that recognize valid language.

The Parser is the part of the compiler which recognizes correct token sequences.

It is a **Push-Down Automaton** (PDA).

Regular Expressions I

Regular Expressions are a notation that uses character *literals* to match strings in a language.

There are 5 rules

- The empty string, denoted by **ϵ**
- A single character
- Two regular expressions concatenated
- Two regular expressions separated by **|** (Alternation)
- A regular expression followed by ***** (the Kleene star)

Here, **ϵ | *** are metacharacters, meaning that they represent actions for the Regular Expression, and do not match *literal* characters.

Regular Expressions II: Example

```
number   : integer | real
integer  : digit digit*
real     : integer exponent | decimal (exponent | ϵ)
decimal  : digit* (. digit | digit .) digit*
exponent : (e | E) (+ | - | ϵ) integer
digit    : 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

What are valid strings according to this RE?

Regular Expressions III

Note that the REs are greedy.

`foobar321` is always recognized as `foobar321`, never `foo` or `f`.

`3.14159` is always `3.14159`, and never `3` and `.14159`

Regular Expressions IV

REs are widely used, and many *extensions* to the basic set of rules exist

REs are very useful in many contexts.

The program sed on POSIX systems is a useful companion to scripting on *nix systems.

Regular expressions are built into many programming languages, like awk, Java, Javascript, Go, python and many many more.

Regular Expressions V - Extended

`*`	matches zero or more of the preceding regex
`+`	matches one or more of the preceding regex
`\?`	matches zero or one of the preceding regex
`\{i\}`	matches i sequences
`\{i, j\}`	matches between i and j sequences
`.`	matches any character, including newlines
`[list]`	matches any element of list. [aeiou] matches any vowel.
`^`	matches the null character at the beginning of a string
`$`	matches the end of a string

Regular Expressions VI

String substitution is very common as well, and usually uses a notation like this

`'s/abc/cba/'`

Which substitutes the string “abc” with “cba”. Any regex can be used instead of abc or cba. The modifier g at the end means match and replace ALL strings matched by the REs.

`echo <input text> | sed 's/<pattern>/<replacement>/<flags>'`

Flags are typically `g` - global, `i` - ignore case, `[0-9]` (its own regular expression!)

A Machine for Regular Expressions

Machines are made from **parts**… What are the parts?

input - What is processed. What is the input?

What type of object is a program?

position - where is the machine

Sets of characters - valid character literals

Lexer/Scanner

This machine has a name:

Lexer!

Scanner!

The action of this machine is referred to as “scanning” or “lexing”

Lexical Analysis I

Scanning is also called **lexical analysis**.

Lexical - *relating to the words or vocabulary of a language*

Lexeme - *a basic lexical unit of a language*

Scanning may also be called lexing

The scanning process groups input characters into tokens

This lessens the load of the more computationally intensive parsing

Lexical Analysis II

Lexical Analysis:

1. Provides a stream of Tokens to the following stages of translation
2. Detects errors in the input

Scanning typically isn’t performed all at once

The Scanner/Lexer is asked for the next token periodically

Lexical Analysis III - Errors

What does the Lexer do when it encounters an error?

Options:

1. Throw away the current token, which is invalid
2. Skip forward until a character is found that can be a valid token
3. Restart scanning
4. Rely on other stages to handle the error (not my problem!)

Lexical Analysis IV - Examples

Numbers. What constitutes a number in your favorite PL?

Identifiers. What is a valid name in your favorite PL?

Whitespace. What is the meaning of whitespace in your favorite PL?

A Machine for Regular Expressions II - Lexical Analyzer

Machines are made from **parts**… What are the parts?

- input - to process. What type is the input?
- **What type of object is a program?**
- position - where is the machine
- Sets of characters - what can it expect to see

Machines are made from functions… What are the functions?

- advance - move ahead
- peek - what is the part the machine is looking at?
- next and friends - gather the next whole part (token)

Lexical Analyzer I

Lexical Analyzers are the programs which perform lexical analysis

Several ways to design them:

1. Ad-hoc
2. Finite Automata
3. Table Driven

Lexical Analyzer II - ad hoc

Ad hoc: *created or done for a particular purpose as necessary.*

An ad hoc scanner is a handwritten program designed for specific tokens in a language.

Tokens are typically numbers, identifiers, keywords, operators

A scanner should also accept whitespace and comments

```
operators  -> '+' | '-' | '*' | '/' | ';' | '=' | '(' | ')'

id         -> letter (letter | digit )*

num        -> digit* (. digit | digit . | ϵ) digit*
```

Lexical Analyzer III

This type of program generally consists of a functions/methods that advance the scanner through the input

Typically, each type of Token has its own recognition algorithm

Pseudocode for an ad hoc scanner

The IA1 assignment is a partially complete ad-hoc scanner written in Java. The next() method is the main driver for the scanning algorithm.

Lexical Analyzer IV - Table Driven

A scanner can also use a lookup table to determine the next Token

This would also be coupled with an engine for checking the next token

State	whitespace	letter	digit	.	operator	Token
1	6	2	3	4	5	
2		2	2			id
3			3	4		num
4			4			num
5						operator
6	6	1	1	1	1	

Lexical Analyzer V - DFA

In the theory of computation, a Deterministic Finite Automata is a abstract machine that accepts or rejects strings of symbols

It operates with a unique, predictable path for each input

Either ad-hoc or table driven scanners can be visualized this way

Lexical Analyzer VI

Lexers all start from REs

Do we have to write Lexers for every RE?

Why not generate a Lexer using the RE as input?

lex is a tool (1975) that does just this

Context Free Grammars I

Can a Regular Expression match correctly nested parentheses? `(x * (x + y))`

REs are too simple to recognize context free language.

Context Free Grammars (CFG) are rules that can specify a Context Free Language

Context here means that we don't specify conditional application for these rules

CFGs are specified in Backus-Naur Form (BNF), typically with some extensions (eBNF).

BNF was invented for Algol-60 (an ancestor of C)

Context Free Grammars II

The rules used in CFGs are composed of:

- Terminal symbols. Actual Tokens found in the language (recognized by RE)
- Nonterminal symbols. Can *take the form* of other nonterminals or terminals

CFGs are theoretically more powerful than RE because their rules can be self referential.

This implies recursion. Consider the recognizer for matching parenthesis

```
nested: '(' nested ')'
```

This cannot be done with a DFA (regular expression)

Context Free Grammars III

Rules in a CFG are given as pairs, with a Right Hand Side (RHS) and a Left Hand Side (LHS)

Nonterminals appear on the RHS and the LHS, with only one nonterminal on each line on the LHS

Combinations of nonterminals and terminals can appear on the RHS

No terminals can appear in the LHS

Each LHS rule is said to **expand** to the RHS

Each rule is called a **production**

Context Free Grammars IV

Finally, an example:

```
expr      -> id | number | - expr | ( expr )
          |   expr op expr
op        -> + | - | * | /
```

As with RE, | is a metacharacter which indicates disjunction

Context Free Grammars V

Another example: lists of identifiers

```
id_list      -> id id_list_tail

id_list_tail -> , id id_list_tail

id_list_tail :  ϵ
```

Recognizes the same Context Free Language as this one

```
id_list      -> id_list , id

id_list      :  id
```

Context Free Grammars IV

Notice how some nonterminals are really Regular Expressions.

```
expr      -> id | number | - expr | ( expr )
```

Here, `id` and `number` can be any Token of that type from the scanner. The parser does not care about what the actual `id` or `number` is

CFG's are used to **derive** the valid sequences of strings in the language.

This is the mechanism that analyzes a programs syntax for correctness.

Derivations I

Lets practice a **derivation** using the CFG from a previous slide.

A derivation is any sequential expansion of non-terminals from the start to a string with only terminals

The program is `slope * x + intercept`

```
expr      ->   expr op expr
          ->   expr op id
          ->   expr + id
          ->   expr op expr + id
          ->   expr op id + id
          ->   expr * id + id
          ->   id * id + id
```

Derivations II

This is an example of a Right-Most Derivation, because we always selected the right-most non-terminal to expand

The specific algorithm for expansion will be share later

Note we could have also chosen to expand the left-most nonterminal, resulting in a Left-Most Derivation

Derivations III

These derivations can also be displayed graphically, as **parse trees**

Consider the right-most parse tree and left-most parse tree **shown on the board**

The parse trees for this grammar are different! If two distinct parse trees exist for a given string in a CFG, the grammar is said to be ambiguous

Ambiguity is a big problem for building an efficient Parser

The Parser will require another mechanism to drive choice between the expansions.

Also try `prg = 10 - 4 - 3`

Derivations IV

Theoretically, there are an infinite number of CFGs that recognize a given language

Therefore, we can choose (or develop) one that is not ambiguous

A Better CFG for exprs

```
expr      -> term | expr addop term

term      -> factor | term mulop factor

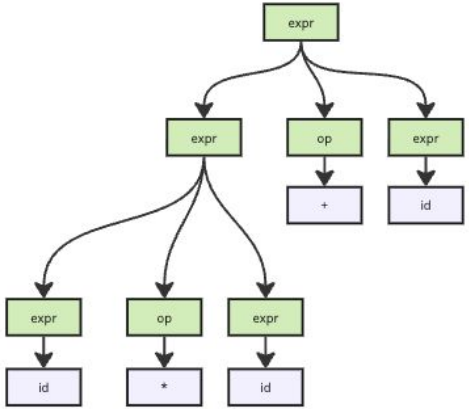
factor    -> id | number | - factor | ( expr )

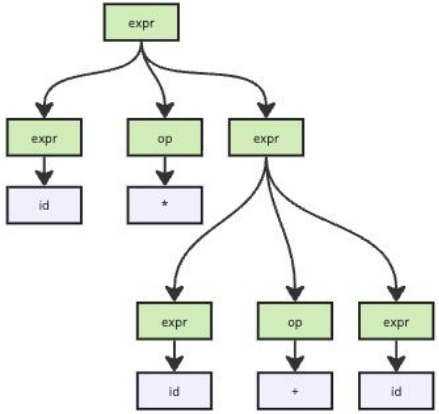
addop     -> + | -

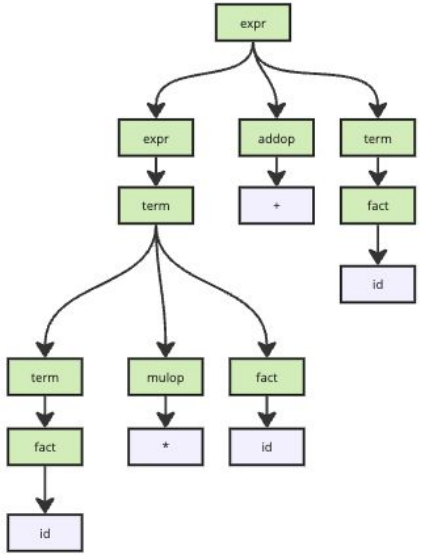
mulop     -> * | /
```

Demonstrate on `slope * x + intercept` with right-most and left-most derivations

Examples


expr
expr
op
expr
+
id
expr
op
expr
id
*
id


expr
expr
op
expr
id
*
expr
op
expr
id
+
id


expr
expr
addop
term
term
+
fact
id
term
mulop
fact
fact
*
id
id

More examples:

3 + 4 * 5

10 - 4 - 3

(1 + 2)

Your turn!

C Examples

Here are some examples in C using a similar grammar:

```
additive_expression
        : multiplicative_expression
        | additive_expression '+' multiplicative_expression
        | additive_expression '-' multiplicative_expression
multiplicative_expression
        : cast_expression
        | multiplicative_expression '*' cast_expression
        | multiplicative_expression '/' cast_expression
        | multiplicative_expression '%' cast_expression
```

Parsers I

Now we'll explore algorithms for creating parse trees.

A program which builds a parse tree is known as a Parser.

The Parser is the engine which drives the compiler. Its "the boss" of the operation.

The Parser calls the Lexer for tokens, building a parse tree piece by piece.

This tree is usually simplified to an Abstract Syntax Tree (AST).

When the parser is used as the engine of the translator, it is known as Syntax-Directed Translation.

Parsers II

CFGs are recognizers for Context Free Languages (CFL)

A parser is the language recognizer.

A parser can tell if an input program is a member of the CFL by using the rules of the CFG

There are infinite CFGs for any given CFL

There are two algorithms that can be used for any CFG:

1. Earley's algorithm (1968)
2. Cocke-Younger-Kasami algorithm (1961)

Both utilize dynamic programming, and are general purpose

Parsers III

These algorithms are much too slow for compilers

They operate in $O(n^3)$ time, where n is the length of the program in characters.

Consider a moderately sized program of ~500,000 tokens

For comparison, GCC is approximately 15 million **lines**

Assuming 1 nanosecond for each ***token***, and no constant factors

$$(5 \times 10^5)^3 = 1.25 \times 10^{17} \text{ ns} = 1.25 \times 10^8 \text{ s} \sim= \textbf{4 years}$$

However, we can restrict the set of accepted CFG to parse in linear time: **O(n)**

LL I

LL Parsers are one such example

LL is for Left to right, Left-most derivation

Input is read left to right, and a left-most derivation is produced

They are also known as top-down or predictive parsers

They are generally considered intuitive to understand

A parse tree is constructed from the root down, based on the current token and a constant number of lookahead tokens

A CFG that can be parsed in this manner using a single lookahead is LL(1)

LL II

Example Grammar for id lists

```
id_list      -> id   id_list_tail
id_list_tail -> , id id_list_tail
              |   ;
```

Parsing can be done by selecting the production which matches the lookahead token

This grammar has 3 productions, which each start with 3 different tokens

`Prg = “A, B, C;”`

LL III

```
Prg = “A, B, C;”
id_list    -> id(A) id_list_tail
           -> id(A), id(B) id_list_tail
           -> id(A), id(B), id(C) id_list_tail
           -> id(A), id(B), id(C);
```

The parser decides if a Token should appear. If it does, it asks the Lexer for one

If the Token doesn’t match the one the parser expects: Syntax Error!

LR I

LR Parsers are another example

LR is for Left to right, Right-most derivation

Also known as bottom-up or shift-reduce parsers

More grammars are LR than are LL

You may find this strategy more intuitive than LL

Tokens are collected as leaves of a parse tree. Parents are created when groups of tokens and other nodes are found to match their productions

LR II

```
id_list      -> id   id_list_tail
id_list_tail -> , id id_list_tail
              |   ;
```

The same grammar as before can be used here. The grammar is both LL and LR

```
Prg= “A, B, C;”
```

LR III `Prg = “A, B, C;”`

-> id(A)
-> id(A),
-> id(A), id(B)
-> id(A), id(B),
-> id(A), id(B), id(C)
-> id(A), id(B), id(C);
-> id(A), id(B), id(C) *id_list_tail*
-> id(A), id(B) *id_list_tail*
-> id(A) *id_list_tail*
-> *id_list*

LR IV

The next token is shifted in continuously, until a production in the grammar is matched and sequences of input can be reduced

More and more abstract fragments of the language are added

You can consider this type of parser to be looking ahead or not

A Right-most derivation is produced in reverse

This is the first type of parser to be widely used

Therefore, right-most derivations are said to be **canonical**

LR V

The grammar shown before isn't ideal for LR parsing

Why?

The entire program was shifted in before any reduction!

This is an improvement:

```
id_list          -> id_list_prefix ;
id_list_prefix   -> id_list_prefix , id
id_list_prefix   -> id
```

This grammar is LR, but not LL

LR VI Prg = “A, B, C;”

-> id(A)
-> *id_list_prefix*
-> *id_list_prefix*,
-> *id_list_prefix*, id(B)
-> *id_list_prefix*
-> *id_list_prefix*,
-> *id_list_prefix*, id(C)
-> *id_list_prefix*
-> *id_list_prefix*;
-> *id_list*

Other Parsers

There are many modifications to these main strategies

LR - SLR, LR(0), LR(1), LALR

LL - SLL, LL(1), full LL, LLR

Most PLs use grammars designed for either LALR or LL(1) parsers

Java uses a LALR parser.

Historically, GCC used a LALR parser generated by Bison

Modern GCC uses hand-written recursive descent LL parsers in the frontend

Parser Design

Let’s design an LL parser for use in our Interpreter

This parser uses the strategy of recursive descent

Recall LL parsers match a production according to a lookahead token

Our first grammar for expressions is not LL, and needs to be converted

The problem is in the left recursion of the `expr` and `term` productions

Parser Design II

```
prg   : expr 'EOF'
expr  : expr addop term
      | term
term  : term mulop fact
      | fact
fact  : id
      | num
      | '(' expr ')'
addop : '+'
      | '-'
mulop : '*'
      | '/'
```

->

```
prg   : expr 'EOF'
expr  : term addop expr
      | term
term  : fact mulop term
      | fact
fact  : id
      | num
      | '(' expr ')'
addop : '+'
      | '-'
mulop : '*'
      | '/'
```

Parser Design III

We don’t cover the algorithm to eliminate left recursion from a CFG in 354

The converted grammar is a middle ground compromise

On one hand, it can be parsed LL

On the other hand, it breaks operator associativity in our parse trees

We’ll need to handle this later, but we’ll ignore it for now

We could have chosen another grammar suggested by the Dragon Book, but that makes evaluation programmatically challenging

Parser Design IV - Dragon Book CFG

```
expr        : term term_tail
term_tail   : addop term term_tail
            | epsilon
term        : fact fact_tail
fact_tail   : mulop fact fact_tail
            | epsilon
fact        : id
            | num
            | '(' expr ')'
addop       : '+'
            | '-'
mulop       : '*'
            | '/'
```

Parser Design V

The parser is a program. Ours is written in Java

Parser is a class. Recursive Descent means to progress from the top to the bottom of the grammar for a given program

Parser methods need to correspond one-to-one with nonterminal symbols in the CFG

It will also need classes for each nonterminal symbol

These classes act as nodes in the parse tree

Methods in the parser class call other methods and consume tokens according to productions from the grammar

Parser Design V

```
expr   : term addop expr
       | term
```

In Parser.java

```
private Expr parseExpr() throws SyntaxException {
   Term term = parseTerm();
   Addop addop = parseAddop();
   if (addop == null) {
       return new Expr(term);
   } else {
       Expr expr=parseExpr();
       expr.append(new Expr(term,addop,null));
       return expr;
   }
}
```

Parser Design VI

IA part 2 breakout

Parse with “10-4-3”

Show program output from IA2

Module 2 Wrap up

Tools for generating scanners and parsers:

- lex - generator for lexical analyzers. Standard unix tool available and designed for integration with C.
- yacc - acronym for “Yet Another Compiler Compiler”. LALR parser generator.
- bison - GNU version of yacc, with many extensions.
- ANTLR - “ANother Tool for Language Recognition”. Capable of generating lexers and parsers. Produces a recursive descent algorithm LL(*) parser.

Chapter 8 of “The Unix Programming Environment” demonstrates their usage, and builds a programming language called “hoc” (higher order calculator). Hoc is now publicly licensed, you can find it here:
https://web.archive.org/web/19991011232359/http://netlib.bell-labs.com/~bwk/hoc.sh
I highly recommend reading at least the first few sections of chapter 8 if you have the book!

Also recommended:
Compilers